Beyond the English Baseline: IterCOMP-X for Vietnamese Multi-Hop Question Answering

Trương Thành Nhân Trần Ngọc Vỹ Trần Nhật Minh

Faculty of Information Technology, VNUHCM - University of Science, Ho Chi Minh City, Vietnam

{25C1505320, 25C1506947, 24C11055}@student.hcmus.edu.vn

Abstract

Training-free prompt compressors are developed and evaluated almost entirely on English, leaving unclear which design choices generalise to a language with different tokenisation, orthography and answer conventions. We reimplement IterCOMP as IterCOMP-X and give it its first evaluation on Vietnamese multi-hop question answering. On the Vietnamese VimQA benchmark and three English datasets, read by a 4-bit Qwen2.5-7B, IterCOMP-X retains 0.19 to 0.35 of the tokens, leads the uncompressed context on two sets while tying on one and trailing on one, and outperforms the token-level baseline LLMLingua-2 by 10.72 to 18.58 F1 points, a lead that survives matching the two at equal retention and holds, with a wider interval, under a second reader from another family. That retention figure is not comparable across languages: on parallel WMT24++ sentences Vietnamese needs 2.39 times as many tokens as English for identical content under `cl100k_base`, so nominal ratios agreeing across tokenisers to within a factor of 1.02 hide a 1.58× spread in billed cost. Our *effective ratio* prices retention in English-equivalent tokens, turning that 0.19 to 0.35 into 0.19 to 0.49. Three further findings are invisible to an English-only evaluation: normalising boolean answers is worth up to 8.67 F1 on Vietnamese and exactly zero in English; removing diacritics, the prevailing convention in informal Vietnamese, costs gold and compressed contexts about 20 F1 each, placing the degradation in the reader; and Vietnamese does not disadvantage the compressor, IterCOMP-X recovering 84% of Oracle on VimQA, more than on any English set.

1 Introduction

Multi-hop question answering aggregates evidence across documents. Retrieval-augmented generation (Lewis et al., 2020) supplies them at two costs: input length grows with their number, and irrelevant passages degrade reader accuracy. We take the documents as given, in the distractor setting these benchmarks provide.

Prompt compression addresses both, by scoring and keeping a subset of the context or by regenerating a shorter one (Jiang et al., 2023; Li et al., 2023; Pan et al., 2024; Xu et al., 2024). The decision is made once, conditioned on the original question, which misses what defines multi-hop QA: evidence for a later hop becomes identifiable only once the earlier hop's is in, so a compressor reading the question alone cannot recognise it. IterCOMP (Yun and Kim, 2026) embeds that reasoning into the loop (Figure 1), but releases no implementation and is evaluated only on English. Whether its design choices transfer is untested: tokenisation, orthography and answer phrasing all differ across languages, and each can break a component tuned on English.

Two of those differences matter, and neither is visible from an English benchmark. Vietnamese writes tone and vowel quality with diacritics a reader may or may not see, and answers polar questions with a word carrying no lexical overlap with the English *yes*, so a reader correct in the wrong language scores zero. And the vocabularies commercial APIs bill on split Vietnamese far more finely than English, so equal retention is not an equal saving.

We reimplement IterCOMP as IterCOMP-X and add the cross-lingual measurement it lacked. Our first contribution is an *effective ratio* pricing retention in English-equivalent tokens, correcting a nominal ratio that understates Vietnamese cost by 2.39× (Section 4.1). Three further findings follow, all in Section 6.3 and none altering the compression loop:

- A boolean-answer normalisation step worth +8.67 F1 on Vietnamese and exactly 0.0 in English, a linguistic fix rather than a metric-inflating one.

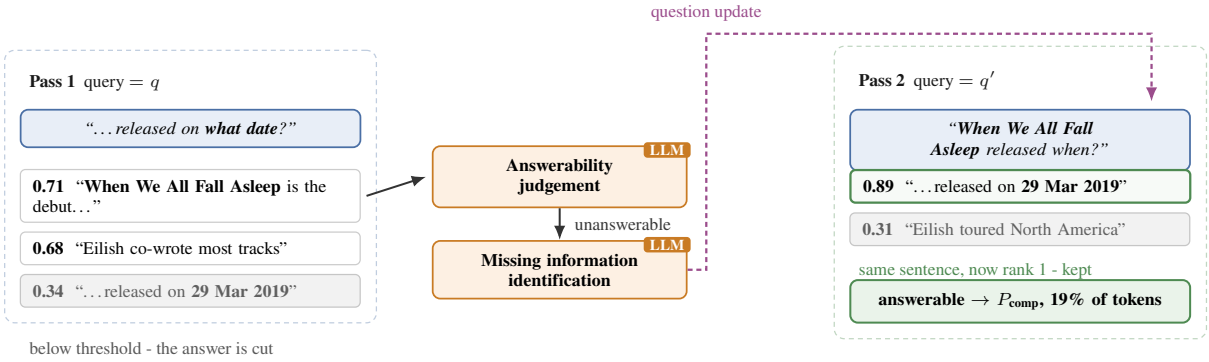


Figure 1: A bridge question through two IterCOMP passes, in English for legibility. The sentence carrying the release date scores 0.34 against the original question and is filtered out; re-scored against the follow-up q' it rises to 0.89 and is kept.

- Evidence that Vietnamese does not disadvantage the compressor: IterCOMP-X retains 84% of Oracle on VimQA against 56–76% in English.
- The first measurement of non-diacritic Vietnamese: stripping diacritics costs about 20 F1, gold and compressed contexts alike.

The compression loop, its scorer, percentile filter, answerability-based stopping and hyperparameters are inherited unchanged; the contribution is one of evaluation and measurement, not a new compression algorithm.

2 Related Work

Prompt compression. Surveys divide the field in two (Li et al., 2025). *Extractive* methods keep a subset of tokens or sentences: LLMLingua (Jiang et al., 2023) drops tokens by perplexity; LongLLMLingua (Jiang et al., 2024) adds query awareness; LLMLingua-2 (Pan et al., 2024) recasts this as token classification, faster and multilingual, the decisive property here; Selective-Context (Li et al., 2023) removes redundant lexical units. *Abstractive* methods regenerate shorter content, chiefly RECOMP (Xu et al., 2024). All of these compress in a single pass, conditioned on the original question.

Iterative retrieval and compression. Self-Ask (Press et al., 2023), IRCoT (Trivedi et al., 2023), ReAct (Yao et al., 2023) and HippoRAG (Gutiérrez et al., 2024) expand the context through a loop over retrieval or reasoning. Two compressors invert that: CompAct (Yoon et al., 2024) condenses documents block by block, carrying a running summary forward, while IterCOMP loops over the reasoning state, using what the reader still cannot answer to decide what to discard. All of this work is evaluated on English.

Tokeniser disparity. Subword tokenisers split non-English text more finely. Ahia et al. (2023) price this against commercial APIs, Somide (2026) extend the accounting to effective context capacity, and Petrov et al. (2023) frame it as unfairness between languages. That literature measures the cost of text; where compression work controls for it at all, it does so by matching budgets in the target tokeniser (Lukauskas, 2026). What remains unproposed is a *reported quantity* carrying the correction, so that published ratios stay comparable across languages; Section 4.1 supplies one.

Vietnamese NLP. Vietnamese is well served at the model level, with PhoBERT (Nguyen and Nguyen, 2020) and ViT5 (Phan et al., 2022), and VimQA (Le et al., 2022) supplies a multi-hop benchmark in the HotpotQA format. XProvence (Mohamed et al., 2026) prunes context in over 100 languages and lists Vietnamese among its transfer targets, but evaluates single-hop QA and reports no per-language retention. To our knowledge no compressor has been evaluated on Vietnamese *multi-hop* QA, and none reports what a Vietnamese context costs under commercial billing tokenisers.

3 Dataset

The three English datasets (HotpotQA (Yang et al., 2018), 2Wiki (Ho et al., 2020), MuSiQue (Trivedi et al., 2022)) are those the original paper used. VimQA (Le et al., 2022) is built from Vietnamese Wikipedia in the HotpotQA format, so our code runs on it unchanged. All four use the distractor setting (Table 1) and development sets, since test answers are held out. Headline results take the full VimQA split and $n = 1{,}500$ for English; supporting measurements take prefixes in file order, so

Dataset	Lang.	#Q	#eval	#psg
HotpotQA	En	7,405	1,500	10
2Wiki	En	12,576	1,500	10
MuSiQue	En	2,417	1,500	20
VimQA	**Vi**	**1,003**	**1,003**	**10**

Table 1: Development-set sizes; VimQA is evaluated in full.

Tokeniser	Vi tok	En tok	Fertility
`cl100k_base`[a]	105.6	44.1	**2.39**×
`o200k_base`[b]	63.3	43.3	1.49×
Qwen2.5 (our reader)	60.2	43.3	1.42×
XLM-R (bge-m3)	57.9	49.7	1.19×
`\w+` (unit of "ratio")	47.6	35.7	1.34×

Table 2: Token fertility over 930 parallel English-Vietnamese WMT24++ pairs (Deutsch et al., 2025), as the mean of per-pair ratios; all 95% intervals are narrower than ± 0.04. OpenAI tokenisers measured with `tiktoken` (OpenAI, 2024). [a]GPT-4, GPT-3.5-turbo; [b]GPT-4o.

carry no selection bias.

4 Method

4.1 Tokeniser fertility and the effective ratio

A compression ratio is read as a saving, which assumes the unit counted costs the same everywhere. It does not. IterCOMP reports a token share without naming its tokeniser; we count with `\w+`, where Vietnamese runs only $1.34\times$ English, so the reported figure looks roughly equitable; billing does not follow this unit. Under `cl100k_base` the same content costs $2.39\times$, under `o200k_base` $1.49\times$, under bge-m3's XLM-R $1.19\times$ (Table 2), and the reader's own Qwen2.5 tokeniser needs $1.42\times$, so the gap is not an artefact of commercial vocabularies.

Write $|x|_{\mathcal{T}}$ for the units a counter $\mathcal{T}$ assigns to a string x, D for the retrieved documents and P_{comp} for the compressed context. The quantity papers report is $\text{Ratio}_{\text{nom}} = |P_{\text{comp}}|_{\mathcal{T}}/|D|_{\mathcal{T}}$. Let $\phi_{\mathcal{T}}(\mathcal{L})$ be the fertility of language $\mathcal{L}$, the mean over parallel pairs of $|x_{\mathcal{L}}|_{\mathcal{T}}/|x_{\text{en}}|_{\mathcal{T}}$. We define the *effective ratio* as what the compressed context costs against an English original of the same content:

$$\text{Ratio}_{\text{eff}} = \text{Ratio}_{\text{nom}} \cdot \phi_{\mathcal{T}}(\mathcal{L}). \tag{1}$$

It leaves English unchanged ($\phi = 1$) and rises as far as the tokeniser penalises the language. The factor is domain-dependent: WMT24++ is news, Wikipedia text gives $\phi \approx 2.01$, and on the evaluated VimQA contexts `cl100k_base` spends 2.13 tokens per `\w+` unit, so VimQA's effective retention lies between 0.41 and 0.49 (Appendix E); we report the parallel-corpus value.

The size of what is hidden. Fixing the compressed contexts (the same 50 VimQA questions, the same IterCOMP-X output) and changing only the tokeniser, the nominal ratio barely moves, 0.236, 0.240 and 0.234 under `cl100k_base`, `o200k_base` and `\w+`, a spread of $1.02\times$. The effective ratio reads 0.564, 0.357 and 0.314 on the same bytes, $1.58\times$ apart between the two tokenisers that bill. Contexts a nominal ratio calls interchangeable are priced almost twice as far apart.

4.2 Iterative multi-hop compression

Each document is split into sentence-level segments $e_{i,j}$. A frozen bge-m3 (Chen et al., 2024) scores each against the query with a dual-aspect relevance, the dense inner product $S_{\text{sem}} = E(q)^{\top} E(e_{i,j})$ combined with the sparse lexical score S_{lex} of M3-Embedding over shared tokens:

$$S_{\text{dual}} = \lambda\, S_{\text{sem}} + (1 - \lambda)\, S_{\text{lex}}. \tag{2}$$

Filtering keeps the segments at or above the k-th percentile of that question's scores, $E_{\text{cand}} = \{e \mid S_{\text{dual}}(q, e) \geq \text{Percentile}(\mathcal{S}, k)\}$, so the threshold adapts to each score distribution. The reader then judges, in one token, whether what has been gathered answers q; the reply is constrained to `ANSWERABLE_YES/NO` and read by substring, so a correct judgement never requires a Vietnamese affirmative. On a negative verdict it writes one follow-up question naming the missing information, which becomes q' for the next pass over the still unselected segments. The loop stops on a sufficient verdict or at `max_iter`; we add a safeguard absent from the original that also stops it once the evidence passes 60% of the source. Neither prompt is published in the original paper; ours are in Appendix B.

5 Experimental Setup

Configuration. We adopt the original paper's hyperparameters, set there with a LLaMA-3-8B (Grattafiori et al., 2024) reader: at most 5 iterations, $\lambda = 0.6$, and percentile $k = 90$ (85 for 2Wiki; VimQA is absent there and takes HotpotQA's 90), plus our 60% evidence cap. Every run uses one reader, Qwen2.5-7B (Qwen Team, 2025) at 4-bit

NF4 on a single Tesla T4, decoding greedily to at most 48 new tokens, so no sampling variance enters the numbers.

Baselines. Raw Documents concatenates everything. Oracle keeps only the gold supporting sentences (the original keeps whole gold documents); it is a reference for selection quality, not a per-question bound, since Raw still beats it on 7 to 10% of questions. LLMLingua-2 (Pan et al., 2024) is the main baseline because its token classifier is natively multilingual and covers Vietnamese, so it runs on all four datasets. Six more run on VimQA at $n = 200$; each carries an English-only component, so they widen the field without comparing like with like. R2C, the original's second-strongest baseline, has no public checkpoint.

Metrics. *F1* is the token overlap between prediction and gold answer under the HotpotQA normalisation, keeping Vietnamese diacritics (stripping them collapses distinct syllables; Appendix A.1); EM orders the methods identically. *F1** adds the boolean normalisation of Section 6.3. *Retention* ($\text{Ratio}_{\text{nom}}$) is the share of the source kept, in `\w+` units, and $\text{Ratio}_{\text{eff}}$ prices it in `cl100k_base` tokens (Eq. 1). *Latency* (s/q) is end-to-end time per question. *Gold-evidence recall* is the share of supporting sentences kept in P_{comp}, *coverage* the share of questions keeping all of them. † marks a 95% paired bootstrap interval (10k resamples) excluding zero.

6 Results and Discussion

6.1 Comparison with compression baselines

Sentence-level compression outperforms token-level LLMLingua-2 by $+10.72$ to $+18.58$ F1 (Table 3), every interval clear of zero, while keeping 0.194 to 0.354 of the context; on VimQA ($n = 200$) it also leads the six other baselines (50.08 against 31.22 down to 7.06). The two are not matched on cost: IterCOMP-X calls a 7B model twice per pass, 3.2 to $7.2\times$ LLMLingua-2's wall-clock time per question and 1.6 to $3.3\times$ the uncompressed context's. Nor is the margin bought with a larger budget: on VimQA and HotpotQA IterCOMP-X already retains *fewer* tokens than LLMLingua-2 and still leads, and on the two sets where it retains more, re-running LLMLingua-2 at IterCOMP-X's retention leaves it behind by $+8.22$ F1 on MuSiQue, CI $[+2.59, +13.99]$, and $+16.79$ on 2Wiki, CI $[+9.90, +23.78]$ ($n = 200$). The margin shows that sentence-level selection keeps evidence token-level scoring discards, not that it is cheaper.

F1*	VimQA	MuSiQue	2Wiki	HotpotQA
n	1,003	1,500	1,500	1,500
Oracle	61.03	44.90	57.65	71.40
Raw	46.87	21.16	35.72	**59.24**
IterCOMP-X	**51.29**	**25.35**	**36.72**	54.58
LLML-2	35.31	14.63	19.24	36.00
Δ vs. Raw	$+4.42^{\dagger}$	$+4.19^{\dagger}$	$+1.00$	$-4.66^{\dagger}$
Δ vs. LLML-2	$+15.98^{\dagger}$	$+10.72^{\dagger}$	$+17.48^{\dagger}$	$+18.58^{\dagger}$
$\text{Ratio}_{\text{nom}}$	0.206	0.340	0.354	0.194
$\text{Ratio}_{\text{eff}}$	0.491	0.340	0.354	0.194
s/q: IterC.-X	4.52	8.73	3.80	3.30
s/q: LLML-2	1.43	1.21	0.69	0.98
s/q: Raw	2.88	2.65	1.15	1.67

Table 3: Main results under Qwen2.5-7B at 4-bit. F1* applies the boolean normalisation of Section 6.3 to every row alike; it is inert on English and lifts VimQA's Raw by $+3.99$ and IterCOMP-X by $+8.67$. Δ rows are IterCOMP-X minus that baseline, † a paired bootstrap interval clear of zero; bold marks the numerically higher of Raw and IterCOMP-X. The Ratio rows are IterCOMP-X's retention; s/q is end-to-end wall-clock seconds per question on one T4.

The original's ordering Oracle $>$ IterCOMP $>$ Raw $>$ LLMLingua-2 (Yun and Kim, 2026) holds on VimQA and MuSiQue, matches without separating on 2Wiki, and inverts on HotpotQA, where Raw leads by 4.66 F1 (Appendix C compares with the original's numbers).

6.2 The ordering is reader-dependent

Oracle exceeds Raw on every dataset while reading 5.7% to 16.4% of the tokens; how large that gap is decides whether compression can exploit it: Oracle/Raw is 2.12 on MuSiQue, 1.61 on 2Wiki and 1.30 on VimQA, the three where IterCOMP-X remains competitive; on HotpotQA it is only 1.21 and Raw is superior.

Substituting the reader (Table 6, Appendix C) shows the dependence from both sides. Shrinking it to 0.5B inverts the HotpotQA loss, but the design uses the reader as its own judge, so the smaller model also compresses; pinning the judge at 7B restores retention to 0.192 and leaves IterCOMP-X ahead of Raw by $+5.70$ F1, CI $[+1.90, +9.63]$, the $+1.96$ it gives up being the weaker judge's extra context. Under Mistral-7B the lead over LLMLingua-2 falls to an interval spanning zero, while the gain over Raw, which Mistral reads far worse, grows: compression helps most where the

reader is weakest on long contexts.

6.3 Effects specific to Vietnamese

Boolean normalisation. It yields $+8.67$ F1 on VimQA at $n = 1{,}003$, interval $[+6.98, +10.47]$, and exactly 0.0 on each English set: a reader asked a Vietnamese question often answers "Yes", correct but scoring 0 against the gold "đúng". The step is gated by the *question*, firing on the closing *phải không/đúng không* frame and never on the gold answer, so it is available at inference and cannot turn a wrong answer right. The drift it repairs is uneven, which is why it is worth $+8.67$ to IterCOMP-X and $+3.99$ to Raw (Appendix A). Without the step the two are level (42.61 against 42.88, interval $[-2.80, +2.27]$), so the gain is a measurement correction; the compression result is that parity itself, reached on a fifth of the tokens where every other compressor falls well below Raw. The lead over LLMLingua-2 does not depend on the step ($+12.99$ unnormalised, CI $[+10.32, +15.68]$).

No sign that Vietnamese disadvantages the compressor. IterCOMP-X retains 84% of Oracle on VimQA (51.29 against 61.03), the highest of the four, against 76%, 64% and 56% on HotpotQA, 2Wiki and MuSiQue. Language and dataset are confounded here (passage count, question structure, difficulty), so we do not read this as Vietnamese favouring the compressor (Appendix F.4).

Diacritic removal. Vietnamese is often typed without diacritics online, a setting no compression paper evaluates. Stripping them is expensive: they carry tone (*ma*, *mà*, *má*, *mả*, *mã*, *mạ* collapse to one form). Both branches lose about 20 F1 (Table 11, Appendix F.2), gold leading by 10.52 F1 with diacritics and 8.48 and 6.75 without, every interval clear of zero at $n = 500$. The loss does not single out the compressor: gold drops 20.70 F1 and the compressed branch 18.66 under `strip`, a difference-in-differences of $+2.04$, CI $[-2.71, +6.79]$, so most of the loss is the reader failing on merged syllables. Evidence selection suffers too: gold-evidence recall falls from 91.0% to 84.9% (-6.1 points, CI $[-8.3, -3.8]$) as the lexical scorer matches syllables that no longer carry tone.

7 Error Analysis

We hand-categorised 50 IterCOMP-X predictions on VimQA (Table 12, Appendix F.3). Most errors belong to the *reader*: Oracle sees the gold evidence yet reaches only 61.03 F1, so 80% of IterCOMP-X's error persists under perfect retrieval. One error in ten is a yes/no answer right in substance and wrong in surface form ("tức là yes" for "đúng"), language confusion (Marchisio et al., 2024) that a surface rule cannot fully catch.

A further **9.8% of VimQA is a single question template** - "is X older than Y?" - on which 44.9% of answers score zero against 37.1% elsewhere, because the task is arithmetic rather than retrieval.

Evidence is rarely what is missing. Measured on the first 500 VimQA questions, P_{comp} keeps 91.0% of the gold sentences and all of them for 83.0% of questions. Of the 67 questions gold context answers and IterCOMP-X misses, only 29 lack a gold sentence; on the other 38 the reader fails with the answer in its context. The gain over Raw sits on questions whose evidence spans two documents ($+6.57$, CI $[+2.64, +10.52]$, $n = 626$); on single-document questions the two tie ($+0.86$, CI $[-2.37, +4.00]$). Two cheap ways to keep more evidence do not raise F1 (Appendix D.2).

8 Limitations and Ethical Considerations

Limitations. Several limits bound this work. A second reader more than halves the margin over the strongest baseline, and the loop's judge calls make IterCOMP-X 3.2 to $7.2\times$ slower than LLMLingua-2. Fertility rests on one language pair and on translated news text (Deutsch et al., 2025), so it holds for the tokenisers we bill against rather than the language at large. Absolute numbers are not comparable to the original's: prompts (ours in Appendix B), reader and retention all differ. The language comparison is confounded with dataset, and the domain is narrow, since VimQA derives from Wikipedia.

Ethics. VimQA's CC BY-NC-SA 4.0 licence confines us to research use; compression is lossy and can drop a negation, so sensitive uses should check outputs against the source.

9 Conclusion

A nominal compression ratio that moves $1.02\times$ across tokenisers hides a $1.58\times$ spread in billed cost, which the effective ratio exposes. IterCOMP-X leads token-level compression on all four datasets and uncompressed context on two, keeps 91% of the gold evidence, and leaves most errors to the reader, at 3.2–$7.2\times$ the latency.

References

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah A. Smith, and Yulia Tsvetkov. 2023. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of EMNLP*.

Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of ACL*.

Daniel Deutsch, Eleftheria Briakou, Isaac Caswell, Mara Finkelstein, Rebecca Galilee, Juraj Juraska, Geza Kovacs, Alison Lee, Jason Rudder, and Markus Freitag. 2025. WMT24++: Expanding the language coverage of WMT24 to 55 languages and dialects. In *Proceedings of WMT*.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 herd of models. Technical report, Meta AI.

Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Proceedings of NeurIPS*.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of COLING*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of ACL*.

Ehsan Kamalloo, Nouha Dziri, Charles Clarke, and Davood Rafiei. 2023. Evaluating open-domain question answering in the era of large language models. In *Proceedings of ACL*.

Khang Le, Hien Nguyen, Tung Le Thanh, and Minh Nguyen. 2022. VIMQA: A vietnamese dataset for advanced reasoning and explainable multi-hop question answering. In *Proceedings of LREC*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of NeurIPS*.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of EMNLP*.

Zongqian Li, Yinhong Liu, Yixuan Su, and Nigel Collier. 2025. Prompt compression for large language models: A survey. In *Proceedings of NAACL*.

Mantas Lukauskas. 2026. Lost in compression: A controlled cross-lingual audit of extractive prompt compressors. Preprint.

Kelly Marchisio, Wei-Yin Ko, Alexandre Bérard, Théo Dehaze, and Sebastian Ruder. 2024. Understanding and mitigating language confusion in LLMs. In *Proceedings of EMNLP*.

Youssef Mohamed, Mohamed Elhoseiny, Thibault Formal, and Nadezhda Chirkova. 2026. XProvence: Zero-Cost Multilingual Context Pruning for Retrieval-Augmented Generation. In *Proceedings of ECIR*, volume 16484 of *Lecture Notes in Computer Science*. Springer.

Dat Quoc Nguyen and Anh Tuan Nguyen. 2020. PhoBERT: Pre-trained language models for vietnamese. In *Findings of EMNLP*.

OpenAI. 2024. tiktoken: A fast BPE tokeniser for use with OpenAI's models. `https://github.com/openai/tiktoken`.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of ACL*.

Aleksandar Petrov, Emanuele La Malfa, Philip Torr, and Adel Bibi. 2023. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Loc Pham, Tung Luu, Thu Vo Nguyen, Minh Nguyen, and Truong Son Nguyen. 2026. VN-MTEB: Vietnamese massive text embedding benchmark. In *Findings of EACL*.

Long Phan, Hieu Tran, Hieu Nguyen, and Trieu H. Trinh. 2022. ViT5: Pretrained text-to-text transformer for vietnamese language generation. In *Proceedings of NAACL Student Research Workshop*.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2023. Measuring and narrowing the compositionality gap in language models. In *Findings of EMNLP*.

Qwen Team. 2025. Qwen2.5 technical report. Technical report, Alibaba Group.

Olaoye Anthony Somide. 2026. The African language tax: Quantifying the cost, latency, and context penalty of tokenizing African languages in frontier LLMs. Preprint.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of ACL*.

Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving retrieval-augmented lms with context compression and selective augmentation. In *Proceedings of ICLR*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of ICLR*.

Chanwoong Yoon, Taewhoo Lee, Hyeon Hwang, Minbyul Jeong, and Jaewoo Kang. 2024. CompAct: Compressing retrieved documents actively for question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.

Jungmin Yun and Youngbin Kim. 2026. IterCOMP: Reasoning-aware adaptive prompt compression for multi-hop question answering. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, San Diego, California.

A Vietnamese-specific Handling and Boolean Normalisation

A.1 Prerequisites for running on Vietnamese

The decisive practical advantage: no model needs replacing - both bge-m3 and LLMLingua-2 are natively multilingual and cover Vietnamese. Three mandatory adjustments all sit in text handling. *Unicode-aware tokenisation*: the tokenising regex must set the Unicode flag, or accented characters (*ế*, *ộ*, *ữ*) fail to count as word characters and are split off from their syllables, rendering EM and F1 meaningless. *Preserving diacritics*: HotpotQA normalisation lowercases, strips punctuation and removes English articles, but stripping Vietnamese tone marks collapses distinct words (*má*, *mà*, *mã*) into one. *Boolean answer normalisation*: about 30% of VimQA questions are polar and answer *đúng* or *không*, detailed below. VimQA is also absent from the original paper, so no k is published; we use $k = 90$ as for HotpotQA since both supply 10 candidate passages.

The scorer inherits the syllable granularity of bge-m3's XLM-R tokeniser, so the lexical term can score a syllable shared by unrelated compounds; we run no word segmenter, leaving the dense term to separate them.

A.2 Boolean answer normalisation

The error analysis (Section 7) shows the reader answering yes/no questions in English ("Yes", "tức là yes") where the gold answer is "đúng". This is code-switching in a small multilingual model (Marchisio et al., 2024) meeting the format brittleness of lexical-match metrics (Kamalloo et al., 2023), on a benchmark whose largest answer class is boolean. VimQA's own normalisation (Le et al., 2022) covers Unicode and diacritics only, not answer semantics. We map a prediction into the Vietnamese lexical set only when the *question* closes with the polar frame (*phải không*, *đúng không*) and the prediction contains an unambiguous affirmative or negative token. The gate reads the question, never the gold answer, so the step is available at inference and cannot turn a wrong answer right.

Table 4 carries its own control: normalisation lifts every method on Vietnamese but changes every method by exactly 0.0 on English, where predictions already match the gold tokens. A generic scoring trick would help both languages; this one is inert outside Vietnamese, which is what distinguishes a language-specific fix from a metric-inflating one. The lift is largest for Oracle and IterCOMP-X and smallest for Raw; Table 5 measures the drift behind it. An early reading at $n = 50$ (+10.0 for IterCOMP-X, $p = 0.01$) over-stated the lift; at $n = 1{,}003$ it is +8.67.

Method	ΔF1 (vi)	95% CI
Raw	+3.99	[+2.89, +5.28]
Oracle	+8.97	[+7.28, +10.77]
LLMLingua-2	+5.68	[+4.29, +7.18]
IterCOMP-X	+8.67	[+6.98, +10.47]

Table 4: Effect of boolean normalisation on VimQA, all $n = 1{,}003$ questions, paired bootstrap (10k resamples), Qwen2.5-7B. Every interval is clear of zero. On each English set every method moves by exactly 0.0.

Reader	Raw	LLML-2	Oracle	IterC.-X
Qwen2.5-7B	3.3%	2.0%	11.6%	12.3%
Mistral-7B	35.8%	91.0%	88.1%	89.6%

Table 5: English answers returned on polar VimQA questions, the drift the normalisation repairs. Both readers run at 4-bit; $n = 302$ polar questions under Qwen and 67 under Mistral at $n = 200$.

The drift is uneven (Table 5), measured on the 30.1% of VimQA that is polar. No property of the context orders the systems: under Qwen the most fragmented context drifts least and the least fragmented most, and under Mistral that ranking dissolves. Quantisation is not the cause either, since both readers run at 4 bits. What survives is the gap between the compressed and raw columns, which holds under both readers, and the normalisation prices it.

A.3 Fertility-corrected compression accounting

Fertility is a named, measured phenomenon (Petrov et al., 2023; Ahia et al., 2023), not our discovery. What is unreported is its consequence for compression *cost*: an equal token ratio in two languages is not equal spend, since the same content bills 2.39× more tokens in Vietnamese under `cl100k_base`. The survey of Li et al. (2025) never mentions language; LLMLingua-2 (Pan et al., 2024) evaluates on Chinese without discussing token counts. Appendix E shows across four writing systems that the penalty is the tokeniser's, not Vietnamese's. We also tested, and *refuted*, a second hypothesis: token-level compression was expected to sever Vietnamese syllables, but was measured to sever **zero** (Appendix F.1).

A.4 Why not replace bge-m3 with a Vietnamese encoder

The obvious improvement - swapping the multilingual encoder for a Vietnamese one - we deliberately do not make. VN-MTEB (Pham et al., 2026) compares 18 models over 41 datasets and finds the opposite of the intuition: bge-m3 scores 39.84 on retrieval against 34.18 for `AITeamVN/Vietnamese_Embedding`. More decisively, Vietnamese-specific encoders are *dense-only* with no sparse head, so substituting one would forfeit S_{lex} and collapse Eq. 2 to its semantic term. VN-MTEB itself evaluates dense retrieval only - to our knowledge, no work has benchmarked hybrid dense-sparse retrieval for Vietnamese.

A.5 A negative result on hyperparameters

Sweeping $\lambda \in [0.0, 1.0]$ and $k \in [70, 95]$ on VimQA *under a 0.5B reader at* $n = 50$ puts the best λ at 0.4 against the paper's 0.6 - but the gap is 0.8 F1, and resolving that would need $n \approx 37{,}000$ questions against VimQA's 1,003. We report a **negative result**: no evidence the paper's hyperparameters need changing for Vietnamese. On that sweep $k = 90$ is also optimal, matching the paper; the 7B sweeps in Appendices D.3 and D.1 put the best k elsewhere, but every one of those gaps is inside its own detection threshold, so the three readings disagree only within noise.

B Prompts Used

The original paper does not release prompt text. We give ours in full so that subsequent work can reproduce our setup exactly. Placeholders in `{...}` are substituted at run time.

B.1 Answerability judgment (Step 2)

```
You are solving a multi-hop question.
Below is the evidence collected so far.

QUESTION: {question}

EVIDENCE SO FAR:
{evidence}

Is the evidence above SUFFICIENT to
answer the question accurately?
Reply with exactly one word:
ANSWERABLE_YES or ANSWERABLE_NO
```

B.2 Follow-up question generation (Step 3)

```
You are solving a multi-hop question but
the evidence is not yet sufficient.

ORIGINAL QUESTION: {question}

EVIDENCE SO FAR:
{evidence}

State ONE short follow-up question
targeting exactly the MISSING information
needed to answer the original question.
Give only the follow-up question, no
explanation.

FOLLOW-UP:
```

B.3 Reader prompt

Shared by *every* compression method, so that comparisons differ only in the context supplied. The prompt is bilingual because we evaluate both English and Vietnamese datasets.

```
Answer the question using ONLY the
context below. Reply with the SHORT
answer only (a few words, no
explanation, no full sentence). If the
context does not contain the answer,
reply: unknown

CONTEXT:
{context}

QUESTION: {question}
ANSWER:
```

C Results at Full Scale

C.1 VimQA at $n = 1{,}003$

The ablations and baselines run at smaller n because each was run when the compute allowed. This appendix gives the largest: all four methods on **all 1,003 VimQA questions** with the same 7B reader, which drops the paired detection threshold from 18.0 F1 (at $n = 50$) to **4.3**.

Method	EM	F1*	ratio
Raw Documents	28.5	46.87	1.000
Oracle	**36.4**	**61.03**	0.057
LLMLingua-2	17.2	35.31	0.316
IterCOMP-X	26.9	51.29	**0.206**

Three paired comparisons, 10k bootstrap resamples:

Comparison	ΔF1	95% CI
IterC.-X − LLML-2	+15.98	[+12.98, +19.01]
IterC.-X − Raw	+4.42	[+1.73, +7.17]
Oracle − IterC.-X	+9.74	[+7.08, +12.42]

C.2 All four datasets, dataset by dataset

The central claim holds on all four. Sentence-level compression beats token-level compression by +10.72 to +18.58 F1, every interval clear of zero. That is the original paper's thesis, and the one result that does not depend on which dataset, reader or sample size we look at.

Reader / data	n	$\text{Ratio}_{\text{nom}}$	Δ Raw	Δ LLML-2
Qwen-7B, HotpotQA	1,500	0.194	$-4.66^{\dagger}$	$+18.58^{\dagger}$
Qwen-0.5B, HotpotQA	500	0.532	$+7.66^{\dagger}$	$+16.36^{\dagger}$
Qwen-7B, VimQA	200	0.193	$+5.92^{\dagger}$	$+11.78^{\dagger}$
Mistral-7B, VimQA	200	0.182	$+23.36^{\dagger}$	$+4.42$

Table 6: Reader substitutions (Qwen2.5 sizes abbreviated), each a separate run with the reader as its own judge; Δ is IterCOMP-X minus the named baseline, † a paired bootstrap interval clear of zero.

The full ordering does not. The paper reports Oracle $>$ IterCOMP $>$ Raw $>$ LLMLingua-2 throughout. We reproduce it on VimQA ($+4.42$, clear) and MuSiQue ($+4.19$, clear at $n = 1{,}500$); on 2Wiki the order matches but $+1.00$ does not separate. Only on HotpotQA does **Raw overtake IterCOMP-X**, by **4.66 F1, interval clear**. An earlier version of this report claimed that "the method ordering reproduces exactly"; that claim was made from MuSiQue alone and is withdrawn.

What tripling the sample resolved. MuSiQue's IterCOMP-X$-$Raw went from $+3.31$ (crossing zero at $n = 500$) to $+4.19$ (clear), and 2Wiki's flipped sign, -2.69 to $+1.00$. Greedy decoding makes the first 500 of the 1,500 reproduce to 0.00 F1, so the $n = 500$ readings were too few, not wrong. Below the detection threshold a difference has only a value, not a reliable sign.

Why, and what it costs the premise. Our reader reads long contexts better than the original's. On HotpotQA Raw scores 59.24 against the paper's 43.63 ($+15.6$ F1 from the reader alone, the largest such gap here) while IterCOMP-X moves only from 51.78 to 54.58. A reader that copes with distractors gains more from being handed everything than from a filtered subset, so compression's *accuracy* advantage evaporates and only its *cost* advantage survives: IterCOMP-X still reads 19.4% of the tokens. This qualifies the premise that irrelevant passages degrade reader accuracy: they do on VimQA and MuSiQue, where Oracle/Raw is 1.30 and 2.12, but on HotpotQA (1.21) and 2Wiki (1.61) the reader recovers enough from the full context that filtering costs accuracy.

C.3 What the move to larger samples cost MuSiQue

MuSiQue is the one dataset we have at three sizes, so it shows what small samples were hiding. Oracle/Raw/IterCOMP-X/LLMLingua-2 F1* read 57.05/32.85/35.11/13.51 at $n = 50$, 49.84/23.43/26.74/15.14 at $n = 500$ and 44.90/21.16/25.35/14.63 at $n = 1{,}500$: the first questions were simply easier than the set. Two readings changed with it: at $n = 500$ IterCOMP-X no longer separated from Raw on English (resolved in its favour at $n = 1{,}500$), and the margin over LLMLingua-2 is larger on Vietnamese ($+15.98$) than on English ($+10.72$), reversing an $n = 50$ reading that had looked like evidence for the advantage growing with benchmark difficulty.

Method	MuSiQue		VimQA
	paper	ours	ours
Oracle	37.51	57.0	53.3
Raw Documents	19.92	32.8	43.8
LLMLingua-2	14.72	13.5	30.7
IterCOMP	**27.36**	**35.1**	**43.0**
ratio	*0.14*	*0.32*	*0.19*
Oracle/Raw	*1.88*	*1.74*	*1.22*

Table 7: F1 from the original paper (LLaMA-3-8B reader) against our first reproduction (Qwen2.5-7B at 4-bit, $n = 50$), plus our Vietnamese result at the same size. This is the earliest comparison; Table 3 supersedes it at scale.

C.4 Comparison with the original paper

Absolute numbers are not comparable across readers, but two checks are consistent with the original (Table 7). First, the method ordering reproduces on MuSiQue. Second, the Oracle/Raw ratio is in range: 1.74 at $n = 50$ and 2.12 at $n = 1{,}500$ against the paper's 1.88. This is a loose check, since our Oracle keeps gold sentences and the original's whole gold documents.

Matching the operating point. A direct $n = 500$ comparison on MuSiQue puts IterCOMP-X 0.62 F1 *below* the paper's (26.74 against 27.36), while Oracle and Raw - which our compression code never touches - sit far above it ($+12.33$ and $+3.51$), our stronger reader alone. That -0.62 is misleading: it compares systems at different operating points, 0.349 retention against 0.14. At $k = 90$ - the paper's setting - our retention on MuSiQue is 0.349 against their 0.14 (Table 8): same knob, same dataset, $2.5\times$ apart, so the difference is in the segment population the percentile ranks over (MuSiQue supplies 20 passages per question and our decomposition yields more candidates). We therefore swept k to find where our retention meets theirs.

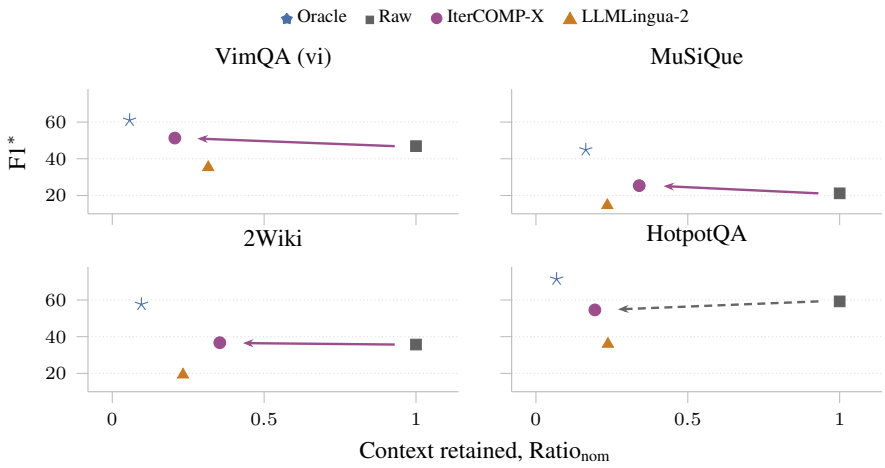


Figure 2: The compression trade-off, one panel per dataset: nominal retention on the horizontal axis, reader accuracy on the vertical, so the top-left region is better. The arrow runs from Raw to IterCOMP-X: up-left on VimQA and MuSiQue, flat on 2Wiki (+1.00, not separating), down-left on HotpotQA (dashed). LLMLingua-2 sits below IterCOMP-X in every panel, though not at matched retention.

k	keep	F1	iters	segs
90 (paper's)	0.330	26.65	3.14	6.3
93	0.287	26.32	3.19	5.4
96	**0.160**	**29.04**	3.00	3.0
97.5	0.160	29.04	3.00	3.0
paper	*0.14*	*27.36*	–	–

Table 8: MuSiQue, Qwen2.5-7B at 4-bit, $n = 200$. $k = 96$ and $k = 97.5$ are identical because the filter floors at three segments per question.

At $k = 96$ we retain 0.160 against their 0.14 and score **29.04** against **27.36**: at comparable compression our reimplementation is +1.68 above the original, not 0.62 below - the deficit at $k = 90$ was an artefact of reading 2.5× more context, which costs accuracy on MuSiQue because the extra passages are distractors. Two limits: the sweep floors at $k = 96$, so we cannot reach their 0.14 exactly, and +1.68 is well inside the detection threshold at $n = 200$, so the honest statement is that the two are indistinguishable at matched compression.

D Ablations and Open Questions

D.1 Ablations at $n = 500$, which refute one of them

We reran the two axes that carry claims at ten times the sample (`ablation_vimqa500_7b.json`).

The bit-identical result was an artefact of $n = 50$. Extra passes changed which segments were retained (0.158 to 0.189) without changing a single answer, to six decimals. An agreement that exact reads as a structural fact rather than a sampling one; a larger sample overturned it: at $n = 500$ the four numbers run 34.9, 35.6, 36.2, 36.1, with mean iterations rising 1.18 to 1.58. Fifty questions were too few to contain the minority on which the ceiling binds - and that minority is narrow, since even at `max_iter` $= 5$ the loop stops after 1.58 passes on average. The remaining finding is weaker: two passes versus five yield +1.2 F1, far inside noise, for 20% more context - an argument for the smaller budget on cost, not accuracy. This is why the iteration budget is not among our findings.

The k-curve reversal is consistent with sampling noise. At $n = 50$, $k = 70$ scored best (38.77) and $k = 90$ worst (33.03), a spread of 5.7 F1. At $n = 500$, $k = 70$ is *worst* (35.9) and the spread has collapsed to 2.3. A ranking that inverts under a tenfold sample increase was never a ranking.

D.2 Can dropped evidence be recovered?

Two cheap remedies were run on the first 500 VimQA questions, paired against the default configuration (`evidence_*_500.json`):

	Recall	Cover.	F1*	Ratio
Default	91.0	83.0	47.47	0.193
+ neighbours ±1	93.0	86.4	48.62	0.276
+ entity q', title anchor	91.1	83.2	46.92	0.197

Pulling in each kept sentence's neighbours raises recall by 2.1 points, CI $[+1.0, +3.3]$, and coverage by 3.4, CI $[+1.4, +5.4]$, but costs 43% more context and moves F1* by +1.15, CI $[-1.59, +3.84]$. Asking the judge to name the bridge entity and adding the best sentences of any document whose

	n=50		n=500				n=500	
m_it	F1	rat.	F1	rat.	it.	k	F1	rat.
1	28.36	.134	31.7	.133	1.00	70	35.9	.431
2	**33.03**	.158	34.9	.160	1.22	80	**38.3**	.320
3	**33.03**	.170	35.6	.174	1.36	85	38.2	.251
4	**33.03**	.181	**36.2**	.185	1.47	90	36.1	.194
5	**33.03**	.189	36.1	.194	1.58	95	38.2	.129

Table 9: VimQA, Qwen2.5-7B at 4-bit; "rat." is the retained fraction, "it." the mean iteration count. At $n = 50$ the F1 column is *bit-identical* from `max_iter` $= 2$ (33.033695 to six decimals); at $n = 500$ it is not. Detection threshold at $n = 500$ is 6.6 F1 paired, so *no* pair in either half separates.

title q' names changes nothing measurable (F1* -0.55, CI $[-1.71, +0.55]$). With recall already at 91%, the reader rather than the compressor is the bottleneck.

D.3 What this reimplementation does not settle

Whether the stopping rule helps - resolved. At $n = 50$, removing the answerability check appeared to raise F1 from 33.03 to 36.64, suggesting the rule *lost* 3.6 F1 - below the 18.0-point paired threshold, so we ran four stopping regimes at $n = 200$ under the 7B reader. A *perfect* oracle stopper scores 49.91 against the paper's 50.08 on VimQA and 25.94 against 25.90 on MuSiQue (-0.17, $+0.04$): the rule incurs no accuracy cost, and the earlier gap was a sampling artefact. Never stopping instead reads 49.7% of the context against 18.1% for the same F1, so the rule's value is *token savings, not accuracy*.

Whether a larger reader lifts the ceiling. Oracle, seeing exactly the gold sentences, still scores zero on 28% of VimQA and 30% of MuSiQue questions; since compression cannot beat Oracle, that fraction bounds what Section 7 attributes to the reader. The 98 age-comparison questions are the identified part; whether a reader at the paper's scale recovers the rest is beyond this GPU budget.

Whether two design choices earn their complexity. Eq. 2 combines dense and sparse signals, yet on VimQA at $n = 50$ either half alone scores *above* the combination (lexical 35.44, dense 35.12, dual 33.03), and even BM25 lands within 0.05 F1 at 32.98; top-3 reaches 37.46 F1 at 16.3% of the context against 33.03 at 18.9% for $k = 90$. Both gaps sit inside the noise floor (separation would need $n \approx 1{,}000$), so we report them as absence of evidence *for* either choice, not evidence against.

Language	`cl100k_base`	`o200k_base`
Hindi	**4.97**×	1.71×
Thai	**3.89**×	1.83×
Vietnamese	**2.49**×	1.52×
Chinese	**1.87**×	1.27×

Table 10: Fertility against English over 400 WMT24++ pairs per language, one writing system each: Devanagari, Thai (unspaced), Latin-plus-tones, Han. All 95% confidence intervals are non-overlapping across languages.

E Token Fertility Measurements

Table 2 (Section 4.1) gives the five counters. The factor is domain-dependent: it is measured on WMT24++ news, and substituting the tokens-per-word of the VimQA and HotpotQA contexts gives $\phi \approx 2.01$, putting VimQA's effective retention at 0.414 rather than 0.491. We report the measured factor, since the word ratio it decomposes into is a property of a translation pair and VimQA has no parallel English; the spread a nominal ratio hides is the same either way. Under the reader's own tokeniser VimQA would read 0.292, not 0.491; $\text{Ratio}_{\text{eff}}$ uses `cl100k_base`, the unit a metered deployment is billed in.

Four writing systems. The claim is about Vietnamese, so it needs a control: if every language were billed alike, fertility would not be worth reporting. Measuring 400 parallel WMT24++ pairs per language (`fertility_multilingual.py`) shows it is not.

Every language exceeds 1.0 with non-overlapping intervals (Table 10), so the mismatch is systematic rather than a Vietnamese quirk; Vietnamese at $2.49\times$ is the *mild* case, Hindi being penalised twice as heavily. The $2.49\times$ here (400 pairs) and the $2.39\times$ in Table 2 (930 pairs) are different samples of the same corpus. On the 200

VimQA contexts a context averages 1,110 `\w+` words but 2,364 `cl100k_base` tokens, so the usual ratios understate billing by about $2\times$.

Does the effective ratio discriminate? A second metric that moved with the first would add nothing. Holding the compressed contexts fixed - the same 50 VimQA questions, the same IterCOMP-X output, only the tokeniser changed - the nominal ratio barely moves while the effective ratio does:

Counter	nominal	effective
`cl100k_base`	0.236	**0.564**
`o200k_base`	0.240	0.357
`\w+`	0.234	**0.314**
spread	$1.02\times$	$\mathbf{1.80}\times$

The effective ratio is $1.58\times$ apart between the two tokenisers that bill and $1.80\times$ apart once the word-level reporting unit is included. Because the contexts are byte-identical across the rows, the whole of that spread is the tokeniser's and none of it the compressor's. A nominal ratio reported on its own conceals all of it, which is the case for adding the second column rather than replacing the first.

F Further Vietnamese Analysis

F.1 Function-word damage

Token-level compression was expected to sever syllables and tone marks (*phở* → *ph*), but on 50 VimQA questions LLMLingua-2 severs **zero** syllables; what it cuts is *function words*. At matched compression the function-to-content loss ratio is unbiased for sentence-level IterCOMP-X but not for token-level LLMLingua-2, and the English control lands in the same place:

n=500	IterC.	LLML-2	gap
VimQA (vi)	$1.01\times$	$1.17\times$	$+0.16$
MuSiQue (en)	$0.99\times$	$1.17\times$	$+0.17$

At $n = 50$ the gaps are larger ($+0.31$, $+0.39$) but shrink with sample size, confirming the bias is a property of compression granularity, not the language; our typological hypothesis gets no support. The damage is concrete - at `rate` $= 0.33$ LLMLingua-2 discards 96.1% of relational and 91.2% of negation particles, and dropping *không* inverts a sentence - yet it barely predicts F1: only at $n = 1{,}003$ does the token-level correlation clear zero ($\rho = -0.067$, 95% CI $[-0.124, -0.004]$), a real but negligible association ($|\rho| < 0.1$), while sentence-level IterCOMP-X stays null. Grammatical damage is a measurable property of compressor output, not a demonstrated cause of the 10.7–18.6 F1 gap, whose mechanism remains open.

Setting	IterCOMP-X	Gold	Δ
`keep`	47.47	58.00	$-10.52^{\dagger}$
`strip`	28.81	37.29	$-8.48^{\dagger}$
`both`	30.61	37.36	$-6.75^{\dagger}$

Table 11: Non-diacritic Vietnamese ($n = 500$, 7B reader). Gold leads in every setting, † every paired bootstrap interval clearing zero ($[-14.40, -6.65]$, $[-12.25, -4.77]$, $[-10.57, -2.94]$); none separated at $n = 200$. `strip` leaves the gold answer accented; `both` strips the reference too. Raw was not run under stripping.

F.2 Undiacriticised Vietnamese

Section 6.3 reports the result and its table (Table 11). **An earlier version of this report claimed the opposite sign**: at $n = 200$ it read IterCOMP's compressed context as beating gold context by 3.78 to 5.82 F1 once diacritics were stripped, and explained that by lexical disambiguation. At $n = 200$ none of those intervals separated from zero; at $n = 500$ gold leads in every setting, with every interval clear of zero. The claim is withdrawn, and the mechanism offered for it with it.

How much ambiguity stripping creates. On the corpus (VimQA contexts and questions, lowercased, NFC), removing tone marks collapses the 13,507 distinct syllables onto 11,526, a 14.7% smaller inventory, and about a fifth of syllable types (20.3%) then share their stripped form with another type. The F1 loss is mostly the reader's: the difference-in-differences between gold and compressed contexts ($+2.04$, CI $[-2.71, +6.79]$) is inside noise. Selection is affected as well: recomputed on full contexts, gold-evidence recall falls from 91.0% to 84.9% and coverage from 83.0% to 71.8% (`exp2_recall_{keep,strip}_500.json`).

A background effect. With diacritics intact IterCOMP-X already beats Oracle on **9.8%** of the 1,003 questions, as Raw does on 10.0%. Oracle is a reference for selection quality, not a per-question bound, which is why gold context does not always win.

Category	0.5B	7B
Fully correct (F1 = 1)	8%	16%
Partially correct (0 < F1 < 1)	34%	32%
Format error on yes/no	**32%**	**10%**
Fully wrong (F1 = 0)	26%	42%

Table 12: IterCOMP error distribution on VimQA ($n = 50$) under two readers. Format errors fall from 32% to 10% with the larger reader, a reader limitation rather than a compression failure.

	Oracle	Oracle/Raw	IterC.-X/Oracle
VimQA (vi)	61.03	**1.30**	**0.84**
HotpotQA (en)	71.40	1.21	0.76
2Wiki (en)	57.65	1.61	0.64
MuSiQue (en)	44.90	2.12	0.56

Table 13: Same reader (Qwen2.5-7B at 4-bit), full-scale runs of Table 3. Oracle/Raw is what perfect evidence selection is worth on the data; IterCOMP-X/Oracle is IterCOMP-X's score as a fraction of the Oracle reference.

F.3 Error categories

F.4 Vietnamese/English decomposition

Vietnamese does not handicap the compressor (Table 13): IterCOMP-X reaches 84% of Oracle on VimQA, the highest of the four. The English sets span 56 to 76%, ordered by Oracle/Raw rather than by language (the reader copes with distractors where that ratio is small), so the disparity is largely the data's; the evaluation does not isolate it further. MuSiQue gives 20 passages per question and VimQA 10.